

Kubernetes Deployment & Troubleshooting Guide

Resolution Log for Skillpulse Microservices Stack on Kind (WSL2)

Target Namespace: skillpulse

Environment: WSL2 / Ubuntu / Kind Cluster

Date: August 2026

1. Executive Summary & Issues Resolved

During the deployment of the **Skillpulse** microservices stack on a local Kubernetes cluster (Kind on WSL2), multiple cascading failure points were encountered across database connectivity, service configuration, WSL virtual machine freeze, and Kubernetes readiness probes. This document provides an end-to-end technical post-mortem, step-by-step diagnostic log, and exact fixes implemented to achieve a fully operational state (`1/1 Running`).

Issue Encountered	Root Cause	Resolution Implemented
<code>Waiting for database... attempt X/30</code>	Hostname string concatenation typo in <code>DB_HOST</code> environment variable.	Corrected <code>DB_HOST</code> in <code>20-backend.yaml</code> to clean service name <code>mysql</code> .
<code>Service "mysql" is invalid: spec.clusterIPs[0]</code>	Immutability violation when switching between Regular & Headless service types.	Deleted existing service/statefulset resources and cleanly re-applied manifests.
<code>-bash: /usr/bin/kind: Input/output error</code>	WSL2 subsystem & LXSS service freeze causing I/O channel breakdown.	Executed <code>wsl --shutdown</code> from Admin PowerShell and restarted LXSS manager.
<code>0/1 READY</code> despite API running	Readiness Probe path mismatch between Deployment spec and Gin router.	Updated <code>readinessProbe.httpGet.path</code> to <code>/health</code> in <code>20-backend.yaml</code> .

2. Diagnostics & Step-by-Step Resolution Workflow

Phase 1: Resolving Hostname Concatenation & DB Connectivity

Symptom: Backend pod was failing readiness checks and restarting repeatedly (CrashLoopBackOff). Inspection using `kubectl describe pod` revealed:

```
DB_HOST: mysqlskillpulse.svc.cluster.local
```

Cause: The namespace was inadvertently concatenated into the service name without a separating dot.

Fix: Modified `20-backend.yaml` environment block:

```
env: - name: DB_HOST value: "mysql" # Fixed typo (changed from concatenated domain string) - name: DB_PORT value: "3306"
```

Phase 2: Fixing Service Immutability Errors

Symptom: When applying updated MySQL manifests, Kubernetes threw: `Service "mysql" is invalid: spec.clusterIPs[0]: Invalid value: ["None"]: may not change once set`.

Fix Commands:

```
# Delete conflicting Service, StatefulSet, and stale PVCs
kubectl delete service mysql -n skillpulse
kubectl delete statefulset mysql -n skillpulse
kubectl delete pvc --all -n skillpulse

# Re-apply clean MySQL specification
kubectl apply -f 10-mysql.yaml
```

Phase 3: WSL2 Subsystem Freeze Recovery

Symptom: Terminal threw `-bash: /usr/bin/kind: Input/output error` and `Error code: Wsl/Service/E_UNEXPECTED` due to memory/disk deadlock in WSL2.

Recovery Commands (Run in Windows PowerShell as Administrator):

```
# Force shutdown all WSL instances
wsl --shutdown

# Restart LxssManager Service
net stop LxssManager
net start LxssManager

# Update WSL components if needed
wsl --update
```

Phase 4: Readiness Probe Path Alignment

Symptom: Logs confirmed `Connected to MySQL database` and `Listening and serving HTTP on :8080`, but pod status remained `0/1 READY`.

Cause: Gin router initialized health check on `/health`, whereas K8s probe requested a different path.

Fix: Configured exact probe specs in `20-backend.yaml`:

```
readinessProbe: httpGet: path: /health port: 8080 initialDelaySeconds: 5 periodSeconds: 5 livenessProbe:
httpGet: path: /health port: 8080 initialDelaySeconds: 15 periodSeconds: 10
```

3. Full Cluster Re-Deployment Sequence

To deploy or re-create the entire environment cleanly from scratch, execute the following sequential pipeline:

```
# 1. Navigate to manifest directory
cd ~/github-actions-demo/k8s

# 2. Re-create Kind cluster
kind delete cluster --name kind
kind create cluster --name kind

# 3. Apply manifests in dependency order
kubectl apply -f 00-namespace.yaml
kubectl apply -f 10-mysql.yaml
kubectl apply -f 20-backend.yaml
kubectl apply -f 30-frontend.yaml

# 4. Verify deployment rollout status
kubectl get pods -n skillpulse -w
```

4. Final Verified Cluster State

All microservices components are healthy, fully initialized, and reporting green status across all nodes:

Pod Name	Ready	Status	Restarts	Service Port
mysql-0	1/1	Running	0	3306/TCP
backend-xxxxxxxx-xxxxx	1/1	Running	0	8080/TCP
frontend-xxxxxxxx-xxxxx	1/1	Running	0	8080:30080/TCP

Verification Outcome

Backend successfully connected to MySQL database engine, HTTP Gin endpoints initialized on port 8080, health checks passing on `/health`, and Kubernetes marked deployment as **1/1 READY**.